

ДИСЦИПЛИНА	Технологии разработки серверных приложений
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям
ПРЕПОДАВАТЕЛЬ	Дворецкий Артур Геннадьевич
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Ссылка на материал:

[https://github.com/dv0retsky/fastapi-](https://github.com/dv0retsky/fastapi-tutorial/blob/main/FAPI5_Async%20Types%20Cookies/FAPI5_Async%20Types%20Cookies.md)

[tutorial/blob/main/FAPI5_Async%20Types%20Cookies/FAPI5_Async%20Types%20Cookies.md](https://github.com/dv0retsky/fastapi-tutorial/blob/main/FAPI5_Async%20Types%20Cookies/FAPI5_Async%20Types%20Cookies.md)

Практическое занятие №5: Дополнительные типы данных, асинхронность и параметры Cookie

На данном занятии мы рассмотрим дополнительные типы данных, немного затронем асинхронность при обработке запросов, а также посмотрим на различные типы файлов cookie, которые можно использовать для повышения функциональности и безопасности ваших приложений **FastAPI**.

Дополнительные типы данных

До сих пор для аннотации типов вы использовали простые типы данных, такие как:

- `int`;
- `float`;
- `str`;
- `bool`;
- и другие стандартные типы **Python**.

Однако в **FastAPI**, кроме стандартных типов данных, вы можете использовать и более сложные типы. Это позволяет более точно моделировать данные и работать с ними в реальных приложениях. Ниже приведены дополнительные типы, которые можно использовать:

UUID (Universally Unique Identifier)

Тип **UUID** используется для создания уникальных идентификаторов, которые часто применяются для идентификации объектов в распределенных системах.

Пример:

```
from uuid import UUID

@app.get("/items/{item_id}")
```

```
async def get_item(item_id: UUID):  
    return {"item_id": item_id}
```

`datetime.datetime`, `datetime.date`, `datetime.time`

Эти типы используются для работы с датами и временем.

- `datetime.datetime` - хранит и дату, и время.
- `datetime.date` - хранит только дату.
- `datetime.time` - хранит только время.

Пример:

```
from datetime import datetime, date, time  
  
@app.get("/datetime/")  
async def get_datetime(dt: datetime, day: date, t: time):  
    return {"datetime": dt, "date": day, "time": t}
```

frozenset

Тип `frozenset` представляет собой неизменяемое множество. Этот тип полезен, когда вам нужно работать с коллекциями, которые не должны изменяться.

Пример:

```
@app.get("/unique-items/")  
async def get_unique_items(items: frozenset):  
    return {"unique_items": items}
```

Decimal

`Decimal` используется для работы с числами с фиксированной точностью, что важно в задачах, где требуется высокая точность (например, в финансовых расчетах).

Пример:

```
from decimal import Decimal  
  
@app.get("/price/")  
async def get_price(price: Decimal):  
    return {"price": price}
```

bytes

Тип `bytes` используется для работы с бинарными данными, например, с изображениями или файлами.

Пример:

```
@app.post("/upload/")
async def upload_file(file: bytes):
    return {"file_size": len(file)}
```

Применение Annotated из модуля typing

Для более сложных случаев можно использовать `Annotated` из модуля `typing` (или из `typing_extensions` для Python версий ниже 3.9). Это позволяет добавлять дополнительные метаданные к типам данных, например, ограничения или параметры для валидаторов.

Пример:

```
from typing import Annotated
from fastapi import Query

@app.get("/items/")
async def get_items(
    query: Annotated[str, Query(min_length=3, max_length=50)]
):
    return {"query": query}
```

В этом примере с помощью `Annotated` мы добавляем ограничения на длину строки параметра `query`.

FastAPI поддерживает не только базовые типы данных, но и более сложные, такие как `UUID`, `datetime`, `Decimal`, `frozenset`, и другие. Знание этих типов и их правильное использование может помочь вам строить более точные и надежные **API**, что особенно важно в реальных приложениях.

Введение в асинхронное программирование в FastAPI

Асинхронное программирование — это парадигма, которая позволяет выполнять задачи независимо от основного потока выполнения программы, не блокируя его. В отличие от синхронного программирования, где задачи выполняются последовательно (одна за другой, с ожиданием завершения каждой), асинхронное программирование позволяет инициировать выполнение задачи и продолжать работу, не дожидаясь её завершения.

Это особенно полезно для операций, которые могут занимать значительное время, таких как запросы к сети, чтение/запись файлов или взаимодействие с базами данных. Асинхронность повышает эффективность и отзывчивость приложения, особенно в средах с ограниченными ресурсами или при работе с большим количеством задач, которые могут выполняться параллельно.

Концепции асинхронности в Python

В Python асинхронное программирование достигается с помощью библиотеки `asyncio`, которая обеспечивает поддержку определения асинхронных функций и управления асинхронными задачами.

В **Python** для создания асинхронных функций используется ключевое слово `async`, а для вызова асинхронных операций — `await`. Это позволяет писать код, который может выполнять задачи параллельно, не блокируя основной поток программы.

Пример:

```
import asyncio

async def my_task():
    print("Start task")
    await asyncio.sleep(1)
    print("End task")
```

В этом примере мы создаем асинхронную функцию `my_task`, которая будет приостановлена на 1 секунду с помощью `await asyncio.sleep(1)`, но при этом не блокирует основной поток выполнения программы.

Асинхронная поддержка в FastAPI

FastAPI полностью поддерживает асинхронное программирование с использованием **Python's `asyncio`**. Это позволяет вам определять асинхронные обработчики маршрутов, выполнять асинхронные запросы к базе данных и использовать асинхронные **HTTP-клиенты**, такие как `httpx`, для выполнения запросов к внешним **API**.

Пример асинхронного обработчика маршрута:

```
import asyncio
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def read_root():
    await asyncio.sleep(1) # эмуляция длительной операции
    return {"message": "Hello, World!"}
```

В этом примере обработчик маршрута использует `async` и `await`, чтобы приостановить выполнение на 1 секунду (как если бы это была длительная операция, например, запрос к базе данных), не блокируя другие запросы.

FastAPI автоматически управляет асинхронными запросами и задачами, делая ваш **API** более производительным и масштабируемым.

Преимущества асинхронного программирования в FastAPI

- **Повышенная производительность:** Асинхронный код позволяет выполнять несколько операций параллельно, что особенно важно при работе с медленными I/O-операциями (например, запросами к базе данных или внешним API).
- **Меньше блокировок:** Асинхронные задачи не блокируют основной поток приложения, что позволяет обрабатывать больше запросов одновременно.
- **Более эффективное использование ресурсов:** Взаимодействие с внешними сервисами или базами данных, которое может занимать много времени, будет происходить без блокировки других запросов.

С помощью асинхронных обработчиков маршрутов и асинхронных библиотек в **FastAPI** можно создавать высокопроизводительные и масштабируемые приложения.

Асинхронные обработчики маршрутов и фоновые задачи

Чтобы создать асинхронный обработчик маршрута в **FastAPI**, используется синтаксис `async def`. Асинхронный обработчик маршрута может содержать выражения `await`, которые указывают точки, в которых функция приостанавливает выполнение других задач, пока она ожидает завершения асинхронной операции, такой как запрос к базе данных или **HTTP-запрос**.

Асинхронные обработчики маршрутов особенно полезны для обработки задач, связанных с вводом-выводом (**I–O Bound**), таких как выполнение сетевых запросов, чтение и запись файлов или выполнение запросов к базе данных. Вместо того чтобы последовательно ожидать завершения каждой операции ввода-вывода, асинхронные обработчики могут инициировать несколько операций ввода-вывода одновременно, позволяя приложению выполнять другие задачи в ожидании завершения операций ввода-вывода.

Фоновые задачи в FastAPI

FastAPI также поддерживает фоновые задачи, которые выполняются после отправки ответа клиенту. Фоновые задачи полезны для обработки задач, которые не должны блокировать основной цикл запроса-ответа, таких как отправка электронных писем, обновление аналитики или выполнение дополнительной неблокирующей обработки.

Чтобы использовать фоновые задачи в **FastAPI**, можно определить функцию фоновой задачи с использованием класса **BackgroundTasks** и включить её в качестве параметра в обработчик маршрута.

Пример:

```
from fastapi import BackgroundTasks, FastAPI

app = FastAPI()

def write_notification(email: str, message=""):
    with open("log.txt", mode="w") as email_file:
        content = f"notification for {email}: {message}"
        email_file.write(content)

@app.post("/send-notification/{email}")
```

```
async def send_notification(email: str, background_tasks:
BackgroundTasks):
    background_tasks.add_task(write_notification, email, message="some
notification")
    return {"message": "Notification sent in the background"}
```

В этом примере, когда клиент отправляет запрос на маршрут `/send-notification/{email}`, ответ отправляется сразу, но задача отправки уведомления (запись в файл) выполняется в фоновом режиме.

Для более гибкого и функционального управления фоновыми задачами в реальных проектах рекомендуется использовать библиотеку **Celery**, которая предоставляет более мощные возможности для планирования и выполнения фоновых задач. Про **Celery** можно прочитать [здесь](#).

Параметры Cookie

Cookie — это небольшие текстовые фрагменты данных (пара «имя=значение»), которые отправляются сервером и сохраняются браузером пользователя. При каждом запросе к тому же сайту браузер автоматически прикрепляет соответствующие сохранённые **cookie**. Благодаря этому сервер может «помнить» пользователя: например, хранить состояние авторизации, настройки интерфейса или содержимое корзины. **Cookie** обычно используются для управления сессиями, персонализации и сбора статистики.

Что такое cookie и как они работают

Cookie — это не совсем «файлы» в привычном понимании, а скорее небольшие фрагменты данных. По стандартам **HTTP** сервер передаёт браузеру строку заголовка **Set-Cookie**, в которой прописывается имя, значение и параметры **cookie**. Браузер сохраняет эту информацию (например, в виде файла или записи в своей базе данных **cookie**). После этого при каждом последующем запросе к тому же домену браузер включает этот **cookie** в заголовок **Cookie** запроса. Благодаря этому сервер узнаёт, что запрос от того же браузера, и может «привязать» его к предыдущим действиям пользователя. Например, после успешной авторизации сервер установил **cookie-сессию**, и при следующем запросе браузер автоматически отправит этот **cookie** вместе с запросом.

Браузер управляет **cookie** автономно: вы можете посмотреть их список и содержимое в инструментах разработчика (**DevTools**) вашего браузера (обычно на вкладке Application или Storage). Также **cookie** можно устанавливать на клиенте с помощью **JavaScript** через свойство `document.cookie`. Например:

```
// справочно: установка и чтение cookie в браузере
document.cookie = "username=Ivan; expires=Fri, 01 Jan 2030 00:00:00 GMT;
path=/";
console.log(document.cookie);
```

Обратите внимание, что некоторые параметры cookie ограничивают доступ. Флаг **HttpOnly** делает **cookie** невидимыми для **JavaScript** (они не будут возвращаться при чтении `document.cookie`), что

помогает защититься от **XSS-атак**. Флаг **Secure** указывает браузеру отправлять cookie только по защищённому **HTTPS-соединению**. Начиная с 2020-х годов появился параметр **SameSite**, который регулирует отправку cookie при перекрёстных (межсайтовых) запросах и помогает бороться с **CSRF-атаками**.

В целом схема работы выглядит так: сервер в ответ на запрос добавляет **Set-Cookie: name=value**, браузер сохраняет это, а затем при следующих запросах автоматически добавляет **Cookie: name=value**. При необходимости сервер может обновить cookie, отправив новый заголовок **Set-Cookie** с тем же именем и новым значением.

Типы файлов cookie

Существует несколько категорий cookie в зависимости от их назначения и параметров хранения:

- **Сессионные (session) cookies.** Это временные cookie, которые существуют только пока открыто окно браузера. У них не устанавливается дата истечения — после закрытия браузера такие cookie автоматически удаляются. Сессионные cookie часто используются для хранения идентификатора сессии авторизации: сервер после входа пользователя устанавливает **cookie-сессию** с уникальным **ID**, а браузер посылает его на сервер при каждом запросе, чтобы сервер «узнавал» пользователя.
- **Постоянные (persistent) cookies.** Такие cookie имеют явно указанный срок жизни (**expires** или **max-age**) и остаются на устройстве после закрытия браузера. Они передаются серверу при каждом новом посещении сайта до указанной даты. Постоянные cookies иногда называют «следающими»: рекламодатели и аналитические сервисы используют их для отслеживания предпочтений пользователя в течение длительного времени. С другой стороны, их же можно применять в «мирных» целях — например, чтобы запоминать выбор языка или содержимое корзины между визитами.
- **Аналитические и трекерские cookies.** Это cookie, которые устанавливают сторонние сервисы (например, **Google Analytics** или рекламные сети) для сбора статистики о поведении пользователей. Они могут проследить переход пользователя по разным сайтам (через механизм третьих сторон) и строить на основе этого анонимные профили для таргетированной рекламы. Эти cookie не нужны для корректной работы сайта, а используются для маркетинга и аналитики. Большинство современных браузеров позволяют блокировать сторонние (третье) cookie или запрашивать согласие пользователя.
- **Другие категории.** Часто выделяют функциональные cookies (например, для запоминания настроек интерфейса) или технические («необходимые» cookies, которые без них сайт не будет функционировать). Основное отличие аналитических cookie от необходимых в том, что первые служат для сбора статистики, а вторые — для обеспечения самой работы сервиса (например, сохранения сессии).

Параметры Cookie в FastAPI

FastAPI упрощает работу с **cookie-параметрами** — их можно объявлять в функциях маршрутов так же, как и параметры пути или параметры запроса. Для этого импортируется специальный класс **Cookie** из **fastapi** и задаётся параметр функции с его помощью. Например:


```
from fastapi import Cookie, FastAPI

app = FastAPI()

@app.get("/items/")
async def read_items(ads_id: str | None = Cookie(default=None)):
    return {"ads_id": ads_id}
```

В этом примере при запросе `GET /items/` **FastAPI** автоматически извлечёт значение cookie с именем `ads_id` и передаст его в функцию. Если такого `cookie` нет, переменная `ads_id` получит `None`. Таким образом работа с `cookie` в **FastAPI** полностью аналогична работе с другими параметрами запроса (`Path`, `Query` и т.д.).

FastAPI поддерживает несколько типов cookie: **обычные**, **защищённые** и **подписанные**.

- **Обычные cookies** — стандартные `cookie`, которые хранятся в браузере и отправляются на сервер при каждом запросе. Их можно использовать для хранения любых данных, например состояния сессии или настроек пользователя.
- **Защищённые cookies** — это обычные cookie, которым дополнительно устанавливается флаг `Secure`. Такой `cookie` браузер будет передавать серверу только по **HTTPS**, что предотвращает перехват данных по незашифрованному соединению. Также часто используется флаг `HttpOnly`, который запрещает доступ к `cookie` из **JavaScript**, защищая от **XSS-атак**.
- **Подписанные cookies** — содержат специальную цифровую подпись, которая позволяет проверять целостность данных. Сервер может подписать значение `cookie` своим секретным ключом, и при каждом запросе сверять подпись. Если злоумышленник попытается изменить значение `cookie`, подпись не совпадёт, и сервер отвергнет такие данные. Это гарантирует, что `cookie` не подменялись клиентом после установки сервером.

Для установки `cookie` сервером в ответе обычно используется **HTTP-заголовок** `Set-Cookie`. В **FastAPI** (через **Starlette**) это можно сделать, например, с помощью объекта `Response` и метода `set_cookie`. Сервер генерирует заголовок `Set-Cookie: name=value;` атрибуты, и когда браузер получает такой ответ, он сохраняет `cookie` с заданными параметрами. В дальнейшем браузер будет отправлять этот `cookie` назад серверу автоматически при запросах к соответствующему домену.

Таким образом, **FastAPI** позволяет легко получать и устанавливать `cookie` через параметры функций и методы ответа, а ваша основная задача — выбрать нужные атрибуты (`срок жизни`, `Secure`, `HttpOnly`, `SameSite` и т.д.) в зависимости от сценария. Это даёт удобный и простой механизм для управления состоянием и безопасностью через **HTTP Cookie**.

Примечание: для фронтенд-разработчиков стоит помнить, что при отправке запросов с `cookie` через `fetch` или `XMLHttpRequest` нужно включать опцию отправки учётных данных (например, `fetch(url, { credentials: "include" })`), чтобы браузер действительно прикреплял `cookie`. Кроме того, в браузере можно увидеть установленные `cookie` в инструментах разработчика (**DevTools**) и убедиться, что они содержат нужные атрибуты.

Доступ к файлам cookie

FastAPI упрощает доступ к файлам **cookie** в запросах. Вы можете извлекать данные **cookie** и работать с ними в своих обработчиках маршрутов точно так же, как с любыми другими параметрами запроса.

Для получения куков на сервере используется класс **Cookie** из **fastapi**.

Пример:

```
from fastapi import FastAPI, Cookie

app = FastAPI()

@app.get("/")
def root(last_visit = Cookie()):
    return {"last visit": last_visit}
```

В этом примере, параметр **last_visit** в обработчике маршрута будет содержать значение cookie с ключом **last_visit**, если оно существует в запросе. Если куки с таким ключом нет, значение будет равно **None**.

Важно помнить, что название переменной в обработчике маршрута (в примере **last_visit**) должно соответствовать ключу **cookie**, который вы хотите извлечь.

Установка файлов cookie

В **FastAPI** вы можете установить файлы **cookie** в ответе, используя метод **set_cookie** в параметре **Response**. Этот метод позволяет вам задать имя файла cookie, его значение и дополнительные параметры, такие как домен, путь, срок действия и параметры безопасности.

Пример установки **cookie**:

```
from fastapi import FastAPI, Response

app = FastAPI()

@app.get("/set-cookie")
def set_cookie(response: Response):
    response.set_cookie(key="user_id", value="12345", max_age=3600,
    httponly=True)
    return {"message": "Cookie has been set!"}
```

В этом примере устанавливается **cookie** с именем **user_id** и значением **12345**. Также указывается срок действия cookie **max_age=3600** (1 час) и флаг **httponly=True**, что делает cookie доступной только для сервера и недоступной через **JavaScript**, повышая безопасность.

Метод **set_cookie** позволяет настраивать следующие параметры:

- **key**: имя **cookie**.

- **value**: значение **cookie**.
- **max_age**: срок действия **cookie** (в секундах).
- **expires**: дата и время истечения срока действия **cookie**.
- **path**: путь, с которым **cookie** будет доступна.
- **domain**: домен, с которым **cookie** будет доступна.
- **secure**: если **True**, **cookie** будет передаваться только по защищенному соединению (**https**).
- **httponly**: если **True**, **cookie** не будет доступна через **JavaScript**, что повышает безопасность.
- **samesite**: позволяет ограничить, как **cookie** будет передаваться с запросами из других сайтов.

Прочие возможности при работе с cookie

Вы можете указать время истечения срока действия файла **cookie**, используя параметр **expires**, или установить максимальный возраст с помощью параметра **max_age**. Это помогает контролировать срок службы файлов **cookie** и эффективно управлять данными сеанса.

FastAPI предоставляет простой способ удалить файлы **cookie**, установив время их истечения в прошлом. Это дает указание клиенту удалить файл **cookie** из своего хранилища.

Также у класса **Response** есть метод **delete_cookie**, который принимает в качестве аргумента строку (наименование **cookie**) и удаляет её на стороне клиента.

Пример:

```
from fastapi import FastAPI, Response

app = FastAPI()

@app.post("/logout", status_code=204)
async def logout_user(response: Response):
    response.delete_cookie("example_access_token")
    return {"message": "Logged out successfully"}
```

Под капотом этот метод вызывает метод **set_cookie**, устанавливая атрибутам **max_age** и **expires** значение **0**, что эффективно удаляет **cookie**.

Работа с заголовками запросов

Заголовки HTTP — это пара ключ-значение, передаваемая между клиентом и сервером в каждом **HTTP-запросе** и ответе. Заголовки могут содержать важную информацию о запросе или ответе, такую как тип контента, информация о кэшировании, информация о сессии, пользовательские агенты, и многое другое.

Примеры распространённых заголовков:

- **Content-Type**: указывает формат тела запроса или ответа (например, **application/json** или **text/html**).

- **User-Agent:** предоставляет информацию о клиенте, который отправил запрос (например, браузер или мобильное приложение).
- **Authorization:** используется для передачи данных о том, как клиент авторизуется для доступа к ресурсу (например, с помощью **токена** или **логина/пароля**).
- **Accept:** указывает, какой формат контента клиент ожидает получить в ответе от сервера (например, **application/json**).
- **Cookie:** содержит данные, сохранённые на стороне клиента, которые отправляются обратно на сервер.

Зачем нужны заголовки запросов?

Заголовки запросов необходимы для того, чтобы обмениваться метаданными между клиентом и сервером. Например:

- Сервер может использовать заголовок **Accept**, чтобы понять, в каком формате клиент ожидает получить ответ.
- Заголовок **Content-Type** помогает серверу понять, какой тип данных прислан в теле запроса, чтобы правильно его обработать.
- Заголовок **Authorization** необходим для обеспечения безопасности и авторизации пользователя.

Заголовки — это не просто технические детали: они помогают правильно настроить взаимодействие между клиентом и сервером, а также влияют на производительность, безопасность и совместимость приложений.

Как FastAPI работает с заголовками запросов?

FastAPI упрощает работу с заголовками, позволяя извлекать их из запроса и добавлять в ответ. Мы будем изучать, как получить значения из заголовков запроса в обработчике маршрута, а также как настраивать заголовки в ответах, чтобы отправлять клиенту нужную информацию.

Заголовки запросов

HTTP-заголовки — это метаданные, которые сопровождают **HTTP-запрос** или ответ. **FastAPI** позволяет вам получать доступ к заголовкам запросов и работать с ними для извлечения важной информации, такой как токены аутентификации, юзер-агенты и типы контента.

FastAPI позволяет вам получать доступ к заголовкам запросов (**headers**) в рамках ваших функций маршрутизации. Вы можете использовать заголовки для предоставления дополнительной информации или данных авторизации вашему **API**.

Пример:

```
from typing import Annotated
from fastapi import FastAPI, Header

app = FastAPI()

@app.get("/items/")
```

```
async def read_items(user_agent: Annotated[str | None, Header()] = None):
    return {"User-Agent": user_agent}
```

Разберём эту строчку:

- `user_agent` — это имя заголовка, который мы ищем.
- При помощи `Annotated` мы задаём тип данных в заголовке (строка или `None`), соответственно другие типы не пройдут валидацию.
- Также мы аннотируем формат тем, что указываем, что ожидаем именно заголовок (при помощи нашего класса `Header`). То есть с помощью `Annotated` можно аннотировать переменную чем-то другим, кроме её типа, например, классом. В нашем случае это класс `Header`, который указывает, что мы работаем с заголовком, а не обычной переменной.
- Класс `Header` автоматически переводит наш `snake_case` (например, `user_agent`) в формат заголовка — `User-Agent`. То есть **FastAPI** будет искать именно `User-Agent` в запросе.
- Последнее: мы задаём значение заголовка по умолчанию (`= None`), и если заголовок не поступит в запросе, то внутри функции переменная `user_agent` будет равна `None`. Мы можем задать здесь любое другое значение по умолчанию, если захотим.

Подробнее про `Annotated` можно прочитать [тут](#) или в официальной документации **FastAPI**.

Автоматическое преобразование

Класс заголовка (`Header`) в **FastAPI** обладает небольшой дополнительной функциональностью, помимо того, что предоставляют `Path`, `Query` и `Cookie`. Он автоматически обрабатывает некоторые особенности именования заголовков и преобразует их в нужный формат.

Преобразование символов

Большинство стандартных **HTTP-заголовков** используют дефис ("—") в качестве разделителя между словами, например, `User-Agent`. Однако в Python переменная с дефисами недопустима, так как дефис используется для вычитания, а не для именования переменных.

FastAPI решает эту проблему автоматически: когда вы используете заголовки с дефисами, такие как `User-Agent`, то в коде **Python** вы можете работать с ними, используя стиль именования **Python** — с подчеркиваниями (`user_agent`). Это упрощает код и делает его более читаемым.

Пример автоматического преобразования

При использовании заголовка `User-Agent` в запросе вы можете объявить его в **Python** как `user_agent` и **FastAPI** автоматически преобразует это в правильный формат при извлечении заголовка:

```
from fastapi import FastAPI, Header

app = FastAPI()

@app.get("/items/")
async def read_items(user_agent: str | None = Header(None)):
    return {"User-Agent": user_agent}
```

В данном примере `user_agent` — это стандартный Python-стиль, но **FastAPI** преобразует его в `User-Agent` при извлечении из запроса.

Отключение автоматического преобразования

Если по какой-либо причине вам необходимо отключить автоматическое преобразование подчеркиваний в дефисы, можно установить параметр `convert_underscores` заголовка в значение `False`. Однако это необходимо делать только в особых случаях, так как по умолчанию преобразование работает корректно.

Пример отключения:

```
from fastapi import FastAPI, Header

app = FastAPI()

@app.get("/items/")
async def read_items(user_agent: str | None = Header(None,
convert_underscores=False)):
    return {"User-Agent": user_agent}
```

В этом примере заголовок будет извлечен с именем, как оно указано в запросе (с дефисом), без преобразования в стиль с подчеркиванием.

Повторяющиеся заголовки

В **FastAPI** возможно получение повторяющихся заголовков. Это означает, что один и тот же заголовок может быть отправлен несколько раз в запросе и содержать несколько значений. **FastAPI** позволяет вам получить все эти значения и работать с ними как с обычным списком.

Для работы с повторяющимися заголовками нужно объявить параметр с типом `list` в аннотации типа. В результате, все значения повторяющегося заголовка будут собраны в список **Python**.

Например, чтобы объявить заголовок `X-Token`, который может появляться более одного раза, вы можете написать следующий код:

```
from typing import Annotated

from fastapi import FastAPI, Header

app = FastAPI()

@app.get("/items/")
async def read_items(x_token: Annotated[list[str] | None, Header()] =
None):
    return {"X-Token values": x_token}
```

В этом примере `x_token` — это список строк, в котором будут храниться все значения заголовка `X-Token`, если их несколько.

Если вы отправите запрос с двумя значениями для заголовка `X-Token`, например:

```
X-Token: foo
X-Token: bar
```

Ответ от **FastAPI** будет таким:

```
{
  "X-Token values": [
    "bar",
    "foo"
  ]
}
```

FastAPI соберет все значения повторяющегося заголовка и вернет их в виде списка. Обратите внимание, что порядок значений в ответе может отличаться от порядка, в котором они были отправлены в запросе, поскольку заголовки **HTTP** не гарантируют порядок.

Доступ к заголовкам запросов и ответов

Для получения заголовков запроса применяется класс `fastapi.Header`. Например, получим заголовок `User-Agent`:

```
from fastapi import FastAPI, Header

app = FastAPI()

@app.get("/")
def root(user_agent: str = Header()):
    return {"User-Agent": user_agent}
```

Для **отправки** заголовка в конструктор класса `Response` или его наследников параметру `headers` передается словарь, где ключи представляют названия заголовков:

```
from fastapi import FastAPI, Response

app = FastAPI()

@app.get("/")
def root():
    data = "Hello from here"
```

```
return Response(content=data, media_type="text/plain", headers=
{"Secret-Code" : "123459"})
```

Также можно **задать** заголовки с помощью атрибута **headers**, который есть у класса **Response** и его наследников. Данный атрибут фактически представляет словарь, где ключи - названия заголовков:

```
from fastapi import FastAPI, Response

app = FastAPI()

@app.get("/")
def root(response: Response):
    response.headers["Secret-Code"] = "123459"
    return {"message": "Hello from my api"}
```

Made with ❤️ by **dvOretsky**